CareBridge

Healthcare Referral & Settlement Platform

Maintenance & Support Guide

Codebase orientation, routine maintenance, common change procedures, troubleshooting, and the recommended roadmap

DOCUMENT REFERENCE	CB-MSG-2026-009
VERSION	1.0
DATE OF ISSUE	2026-07-18
PREPARED FOR	CareBridge Health
PREPARED BY	CareBridge Engineering Team
CLASSIFICATION	Confidential

Table of Contents

1. Introduction

1.1 Purpose

This guide is written for the engineer who inherits CareBridge. It aims to get that person productive quickly: where things are, how the system is put together, what to be careful of, how to make the changes most likely to be asked for, and what should be improved first.

1.2 Read These First

Order	Document	Why
1	System Architecture & Design Document	Explains the structure and, more importantly, the three algorithms that determine behaviour. Nothing else will make full sense without it.
2	This guide, chapters 2 and 3	Orientation in the codebase and the rules that must not be broken.
3	API Reference	The endpoint catalogue, useful as a lookup rather than a read-through.
4	Deployment & Operations Guide	Configuration and deployment. Needed before the first change reaches production.
5	Security & Compliance Document	The known weaknesses and their remedies, several of which are early roadmap items.

2. Codebase Orientation

2.1 Finding Your Way

The backend follows a consistent path from route to data, so any behaviour can be located mechanically. Given an endpoint, read its router entry, then its controller function, then the services it calls.

```
backend/src/
  server.js          entry point: middleware order, router mounting,
                     socket setup, cron registration, startup sequence

  routes/            14 routers under /v1. Thin by design: path,
                     guards, controller reference. Start here.
  controllers/       16 handlers. Validate, orchestrate, respond.
  services/          business logic. The commission engine is here.
  models/            17 schemas. Validation, indexes, field encryption.
  middleware/        auth.js is 34 lines and worth reading in full.
  jobs/              referralEscalation.js — the per-minute scheduler.
  utils/             crypto, scoring engine, deadlines, email, OTP.
  scripts/           maintenance utilities. Read before running.
  bootstrap/         one-time platform data seeding at startup.

backend/tests/       40 tests. Pure logic, no database.
backend/docs/        design records. See chapter 2.4.
```

2.2 The Files That Matter Most

File	Why it matters
services/ commissionService.js	The single source of truth for dividing a bill. Pure, no database access, and the most heavily tested file in the system. Every money question starts here.
utils/scoringEngine.js	Determines which hospital receives a referral, including the two hard filters that exclude a hospital outright.
jobs/referralEscalation.js	The per-minute escalation logic. Small, and the behaviour it implements is a core platform guarantee.
middleware/auth.js	Thirty-four lines containing the entire authorisation model, including the administrator bypass. Read it before assuming anything about permissions.
services/billingService.js and labBillingService.js	Where billing a case turns into a commission accrual. Both are idempotent.
models/User.js	The identity model, including the owner versus team member distinction and the creation-chain check.
utils/crypto.js	Field-level encryption for patient identity numbers, wired into the schemas transparently.

File	Why it matters
server.js	Middleware order, router mounting order — note that the laboratory administration router is mounted ahead of the general one — and the startup sequence.
frontend/src/App.jsx	The entire route table and the role guard. The map of the client.
frontend/src/utils/api.js	The HTTP client, including the session renewal and request replay logic.

2.3 Naming Conventions

Convention	Meaning
A field name ending in Paisa	An integer number of paisa. Never a rupee value, never a decimal.
Consultant	An external referring doctor. Not to be confused with a hospital doctor.
HospitalDoctor	A doctor employed by a hospital, who receives referrals.
Referral versus LabReferral	Two separate entities with parallel but distinct lifecycles. Laboratory referrals fold admission and billing into one record because laboratories have no admission concept.
WeeklySettlement versus LabSettlement	Structurally identical five-state machines for hospitals and laboratories respectively.
legacy versus additive	The two commission models. Legacy is the nested original and remains the default.

2.4 The Design Records

The backend documentation folder contains three commission design records written in sequence. They are worth reading in order, but only the third describes what the code actually does.

Record	Content	Relationship to the code
Commission system design	The original nested model, and — importantly — the identification that bill-splitting arithmetic had been duplicated across four locations. Contains a fifteen-case edge-case analysis that remains the best guide to how this logic fails.	Historical. Read for the edge-case analysis.
Commission system additive	The proposal to replace nesting with addition, with worked examples including the mixed-consultant settlement requirement.	Historical, superseded by the third.
Commission system v2 per doctor	The stakeholder correction on ownership: the consultant fee belongs to the consultant, the platform charge belongs to the facility. Defines the two scopes, two components, two types, and the resolution order.	This is what the code implements. Read this one.

THE FIRST RECORD IS STILL THE BEST GUIDE TO HOW THIS LOGIC FAILS

Its edge-case analysis covers fixed fees exceeding the deduction, fixed fees exceeding a small bill, mid-agreement rate changes and snapshot integrity, rupee-versus-paisa confusion, concurrency on double-finalisation, and negative figures in settlement totals. Most of these are now covered by tests, but the analysis explains why each matters.

3. Rules That Must Not Be Broken

The following are not style preferences. Each protects against a defect class that is either financially material or silent, and several are enforced by tests that will fail if the rule is broken.

3.1 Money

> **ALL BILL SPLITTING GOES THROUGH THE COMMISSION ENGINE**
> Never compute a commission, a platform charge, or a deduction anywhere else, however trivial the calculation looks. This system already had that defect: the same arithmetic existed in four places, and consolidating it was treated as a correctness requirement. A second implementation will drift, and the resulting figures will be plausible and wrong.

- **All money is integer paisa** — in the database, in transit, and in calculation. Convert at the interface boundary and nowhere else.
- **Snapshot the terms applied** — any new financial record must record the model, types, rates, and amounts used, so the figure remains explainable after terms change.
- **Preserve the governing invariant** — the total deduction must always equal the consultant fee plus the platform charge. This is asserted by tests and is what allows settlements to mix consultants on different models.
- **Keep billing idempotent** — repeating a billing operation must not create a second accrual.
- **Respect the resolution order** — consultant-facility override, then facility terms, then platform defaults. Changing this order silently changes what parties are charged.

3.2 Referral Routing

- **The two hard filters are hard** — a hospital lacking the required department, or with no free bed in the required ward, must be excluded rather than ranked low. Recommending it wastes the response deadline on a hospital that cannot take the patient.
- **Coordinates are longitude first** — reversing them places a facility in the wrong location and silently corrupts every distance calculation for that record.
- **Scoring weights must total 100** — enforced at the data layer. Do not bypass this check.
- **Response deadlines come from urgency** — fifteen minutes, two hours, twenty-four hours. Changing these changes a commitment made to consultants and hospitals.

3.3 Security

- **Every new endpoint gets both guards** — authentication and an explicit role. Adding a route without them creates an unauthenticated endpoint, which is how the existing upload finding arose.
- Never log a patient identity number, a password, or a token. Notification payloads are deliberately stripped of personal data; preserve that.
- Never commit a secret. All configuration comes from environment variables.
- Do not weaken the encryption of patient identity numbers, and do not change the encryption key without a data migration.

3.4 Data

- **Referral codes are allocated atomically through the counter** — never generate one by counting existing records, which is not safe under concurrency.
- Date of birth is the source of truth for age; the age field is derived. Do not write age without the corresponding date of birth.
- Enumerated values are enforced at the schema. Adding a new value requires a schema change, deliberately.

4. Routine Maintenance

4.1 Maintenance Schedule

Frequency	Task
Weekly	Review backend logs for recurring errors and for restarts. Confirm the escalation scheduler is logging its per-minute run.
Weekly	Confirm database backups completed within the retention window.
Monthly	Review dependency security advisories and apply security patches.
Monthly	Review the audit log for unexpected administrative activity.
Quarterly	Restore a backup into a test environment and verify the application runs against it.
Quarterly	Review the administrator account list against the people who should hold unrestricted access.
Quarterly	Review matching weights against referral distribution — the weights shape every recommendation and their effect drifts as the hospital set changes.
Annually	Plan a major dependency upgrade. See section 4.3.

4.2 Applying Dependency Updates

1. Check for advisories in both the backend and frontend projects.
2. Apply patch and minor security updates first; these are usually safe.
3. Run the automated suite. It requires no database and completes in seconds, so there is no reason to skip it.
4. Deploy to a non-production environment and run the short regression pass described in the Test Plan and QA Report.
5. Deploy to production, and watch the logs for the first few minutes.

4.3 Major Version Upgrades

> **THIS PROJECT SITS ON RECENT MAJOR VERSIONS ACROSS THE STACK**
> React 19, Express 5, Mongoose 9, Vite 8, and Tailwind 4 are all current major versions. This is an advantage — the project is not carrying upgrade debt — but it means the next major upgrade of any of them will be a genuine migration rather than a version bump. Plan each individually, upgrade one at a time, and never combine a major upgrade with a feature change in the same deployment.

4.4 Database Maintenance

- Indexes are declared in the schemas and created on startup; no manual maintenance is needed.
- Monitor collection growth. Notifications and audit entries grow fastest and are the first candidates for archival.
- **There is no retention policy** — referrals, patient data, documents, and audit entries are kept indefinitely. If a policy is required, it must be defined and implemented.
- Before any bulk data change, take a backup and verify it, then test the change against a restored copy.

5. Common Change Procedures

The following are the changes most likely to be requested, with the files involved and the pitfalls specific to each.

5.1 Adding a Field to a Referral

1. Add the field to the referral schema with its type and any validation.
2. Accept it in the referral creation and update controllers.
3. Add it to the intake form and the detail view in the client.
4. If it should appear in generated documents, add it to the export service and extend the corresponding test.
5. If it is sensitive, decide explicitly whether it needs field-level encryption, following the pattern used for the identity number.

5.2 Adding an API Endpoint

1. Add the route with both the authentication guard and an explicit role list. Never omit either.
2. Add the controller function. Validate input; put business logic in a service.
3. **If it performs any financial calculation, call the commission engine** — do not calculate inline.
4. If it changes state that another party should learn about, raise a notification and emit the appropriate socket event.
5. Add it to the API Reference.

5.3 Adding a Notification Type

1. Add the type and its title to the notification service.
2. Add its message template, including the messaging channel variant.
3. Call the service from the point where the event occurs.
4. **Ensure personal data is not included in the stored payload** — the service strips it, and that behaviour must be preserved.
5. For a facility event, use the team fan-out so the owner and every team member are reached.

5.4 Adding a Ward Type

1. Add the value to the ward enumeration on the hospital schema.
2. **Add it to the ward lists in the hospital registration form, the bed management screen, and the administrative bed screen** — there are three, and missing one produces an inconsistent interface.
3. Consider whether the scoring engine's required-ward logic should treat it specially, as it does for emergencies and intensive care.

5.5 Changing a Response Deadline

1. The deadlines are defined in one place, in the deadline utility.
2. Change the value there. It applies to referrals created after the change; existing referrals keep the deadline they were given.
3. **Consider the commercial consequence** — these deadlines are a commitment to consultants and a performance standard for hospitals, and hospital response statistics are measured against them.

5.6 Adding a Scoring Factor

> **THIS CHANGE IS LARGER THAN IT APPEARS**
> The weights must total exactly 100 and the constraint is enforced at the data layer. Adding a seventh factor means every existing weight must be redistributed, which changes every recommendation the platform makes. Plan the redistribution deliberately, and expect to explain the change in referral distribution to hospitals afterwards.

1. Add the weight to the scoring configuration schema with a default, and update the validation that checks the total.
2. Implement the factor in the scoring engine and add a test for it.
3. Add it to the administrative scoring screen, including the client-side total check.
4. Redistribute the existing defaults so the total remains 100, and update any deployed configuration.

5.7 Adding a User Role

> **A NEW ROLE TOUCHES EVERY LAYER**
> Adding a role is not a small change: the role enumeration, registration, approval, authorisation, routing, navigation, notification fan-out, and — if the role earns commission — the commission engine and settlement cycle are all affected. Scope it as a project, not a task.

1. Add the role to the user schema enumeration and to the registration handler.
2. Create the role profile schema, following the consultant schema as the closest model.
3. Add role-specific registration, approval handling, and profile screens.
4. Add the routes, the role guard entries, and the navigation for the new portal.
5. Extend the notification fan-out to include the role where relevant.
6. **If the role earns commission, extend the commission engine and settlement handling** — and extend the tests first.

6. Debugging Guide

6.1 Where to Look First

Symptom area	Start here
A money figure is wrong	The payout record's terms snapshot. It records exactly what was applied. Then the commission engine and its tests, which are the specification for the arithmetic.
A referral went to the wrong hospital, or to none	The scoring engine's two hard filters, then the hospital's departments and bed counts. Stale bed counts are the most common cause.
A referral did not escalate	The scheduler logs. If the per-minute run is absent, the job is not running.
A user cannot sign in	Account status and email verification state, in that order. Both refuse sign-in with distinct messages.
A permission is wrong	The authorisation middleware. Remember the administrator bypass — an administrator reaching an endpoint is not evidence that the role list is wrong.
A screen does not update live	The socket address configuration first — it must not include the version prefix — then whether the client joined the expected room.
A notification was not received	The notification service call site, then the channel. Channel failures are independent and do not fail the operation, so a missing message may leave no error in the main flow.
A document is malformed	The export service and its tests, which cover all eight document types including attachment handling.

6.2 Reproducing a Financial Question

Most disputes about a figure are answered without running anything. The payout record carries the model, both component types, both rates, both fixed amounts, and the resulting figures. Read it, and it will normally be evident whether the figure is correct under the terms that applied at the time — which is frequently different from the terms that apply now.

If the figure is genuinely wrong, reproduce it as a test case against the commission engine rather than by inspection. The engine is pure, so a failing test can be written in a few lines and becomes permanent protection against the same defect.

6.3 Useful Checks

```
# Run the full suite (no database required)
cd backend && npm test

# Run one suite
node --test tests/commissionService.test.js

# Confirm the API is up
curl https://<backend-host>/

# Confirm the scheduler is running
# -> look for the per-minute escalation entry in the platform logs
```

7. Known Issues and Technical Debt

A consolidated list of everything recorded across this documentation package, so that a maintainer has one place to look. Each entry states the consequence, not merely the fact.

7.1 Should Be Addressed First

Item	Consequence	Remedy
The file upload endpoint has no authentication guard	The only unauthenticated write endpoint besides the signature-verified payment callback.	Apply the existing guard. No client change needed.
The superseded wallet crediting module remains present	The only commission arithmetic outside the tested engine. A future maintainer could reconnect it by mistake.	Delete the module and the settings belonging only to it.
No integration tests for the settlement state machine	A regression could release payouts against an unverified receipt — a financial control failure.	Add integration tests. Highest-value test target in the system.
No test for the escalation scheduler	A core guarantee whose failure is silent.	Add a test, and an operational alert on the job.
Rate limiting covers only authentication	The rest of the interface is unprotected against automated abuse.	Apply a global limit.

7.2 Scaling Blockers

Item	Consequence	Remedy
One-time-password state in process memory	Codes lost on restart; codes fail intermittently across multiple instances.	Move to a shared cache.
The escalation scheduler runs in the API process	A second instance would run it twice, risking double escalation.	Extract to a worker or add a distributed lock.
No socket adapter	With multiple instances, an event emitted on one would not reach a client connected to another.	Add a shared adapter, at the same time as the replica increase.

7.3 Consistency and Clarity

Item	Consequence	Remedy
Hospital and laboratory settlement endpoints have different path shapes	Two identical state machines reached through differently structured paths.	Align in a future interface version.

Item	Consequence	Remedy
Client data fetching uses two patterns	Two conventions for the same job.	Migrate remaining screens to the query cache.
Two screens open a redundant socket connection	Those users hold two connections where one would do.	Consolidate onto the shared connection.
Two built screens are unreachable	A messaging console and a landing page exist but are not linked in. Built-but-unreachable code misleads a maintainer.	Route them or remove them.
Two libraries are declared as development dependencies but imported at runtime	Works today because the build includes them, but it is incorrect and could break under a stricter install.	Move to runtime dependencies.
No central error handler	Response shapes are consistent by convention only.	Add a central handler.
Payment and messaging settings are not in the configuration template	A deployer will not know they exist without reading the separate runbook.	Consolidate into the template.
Development scratch scripts are committed at the repository root	Several write to the database. A maintainer could run one against production.	Move into a clearly marked folder or remove.

8. Recommended Roadmap

8.1 Phase 1 — Hardening

Small, well-understood changes that reduce risk. None changes user-visible behaviour, so they can be delivered without user communication.

- Place the file upload endpoint behind authentication.
- Apply a global rate limit and an explicit request body size limit.
- Add a central error handler.
- Remove the superseded wallet crediting module and the settings belonging only to it.
- Move the two runtime libraries out of development dependencies.
- Route or remove the two unreachable screens.
- Consolidate the configuration template so every setting the system reads is documented in one place.
- Add an uptime check with alerting.

8.2 Phase 2 — Test Coverage

Converts the manual acceptance scenarios into automated protection, in descending order of value.

- Integration tests for the settlement state machine, covering every transition including receipt rejection and partial consultant confirmation.
- A test for the escalation scheduler, covering ranked advancement, the emergency geographic fallback, and automatic rejection when no option remains.
- Integration tests for the authentication and authorisation guards, including the administrator bypass so that it is asserted deliberately rather than assumed.
- Controller-level tests for referral creation, status transition, and billing.
- A basic frontend test setup for the highest-traffic screens.

8.3 Phase 3 — Scalability

1. Introduce a shared cache and move the one-time-password store into it.
2. Extract the escalation scheduler to a dedicated worker.
3. Add a socket adapter.
4. Only then increase the replica count.

8.4 Phase 4 — Data Governance

- Define and implement a data retention and purge policy.
- Define a subject access and erasure procedure, with supporting tooling.
- Move uploaded documents to time-limited signed addresses.
- Migrate patient identity encryption to an authenticated mode.

- Consider read logging on patient and clinical records if read accountability is required.

9. Support Handover

9.1 Knowledge Transfer Checklist

Item	Confirmed
Repository access transferred to the client organisation	
Hosting platform access transferred for both frontend and backend	
Database access transferred, and the connection string rotated	
Third-party service accounts transferred — file storage, email, and any optional integrations	
All shared secrets rotated by the client	
Configuration inventory recorded in the client's password manager	
Database backups confirmed enabled, and a restore tested	
Named administrator accounts created and the seeded password changed	
This documentation package received and reviewed	
Outstanding scope in the Handover Report reviewed and a decision taken	

9.2 Reporting a Problem

To make a problem report actionable, include the following. The first three are usually enough to locate the relevant records and log entries.

- The time and time zone the problem occurred.
- The affected user's role and registered email address.
- The referral, settlement, or account code where applicable.
- What was expected and what happened instead.
- The exact wording of any error message.
- Whether the problem is reproducible, and the steps if so.
- The backend logs for the surrounding period.

9.3 A Closing Note to the Maintainer

The most important thing to understand about this system is that its correctness risk is concentrated in one place. Referral routing is visible — if it goes wrong, a hospital notices. Money is not: a commission computed slightly wrongly produces a plausible figure that nobody questions until a party reconciles their accounts, by which point months of settlements may be affected.

That is why the commission engine is pure, why it is the single path for every split, why it carries seventeen tests, and why every payout records the terms that produced it. If you change nothing else

about how this codebase is maintained, keep those four properties intact.